

PyTorch

2026 年 6 月 10 日

1 PyTorch 简介

在本次作业中，你已经编写了大量代码来实现各种神经网络功能。Dropout（随机失活）、Batch Norm（批量归一化）和二维卷积是计算机视觉领域深度学习的一些核心技术。你也努力让自己的代码高效且向量化。

不过，在本次作业的最后一部分，我们将暂时放下你编写的出色代码库，转而使用两个流行的深度学习框架之一——在本部分中，我们将使用 PyTorch。

1.1 我们为什么使用深度学习框架？

- 我们的代码现在可以在 GPU 上运行了！这会让我们模型训练速度快得多。使用像 PyTorch 这样的框架时，你可以利用 GPU 的算力来运行自己定制的神经网络架构，而无需直接编写 CUDA 代码（这超出了本课程的范围）。
- 在这门课中，我们希望你能项目做好使用这类框架的准备，这样你就能更高效地进行实验，而不必手动编写每一个想要使用的功能。
- 我们希望你能站在巨人的肩膀上！PyTorch 是一个非常出色的框架，它会让你的工作轻松很多，而且现在你已经了解了它们的核心原理，就可以放心地使用它们了：)
- 最后，我们希望你能接触到在学术界或工业界可能会遇到的那种深度学习代码。

1.2 什么是 PyTorch？

PyTorch 是一个用于在 Tensor（张量）对象上执行动态计算图的系统，这些张量的行为与 numpy 的 ndarray 类似。它配备了强大的自动微分引擎，省去了手动反向传播的需求。

1.3 如何学习 PyTorch？

我们以前的一位讲师 Justin Johnson 制作了一个非常棒的 [PyTorch 教程](#)。

你也可以在这里找到详细的 [API 文档](#)。如果有 API 文档未涵盖的其他问题，[PyTorch 论坛](#) 是比 Stack Overflow 更好的提问地方。

2 目录

本次作业包含 5 个部分。你将在**三个不同的抽象层次**上学习 PyTorch，这会帮助你更好地理解它，并为期末项目做好准备。

1. 第一部分，准备工作：我们将使用 CIFAR-10 数据集。
2. 第二部分，基础 PyTorch: **抽象层次 1**，我们将直接使用最低级别的 PyTorch 张量 (Tensors)。
3. 第三部分，PyTorch 模块 API: **抽象层次 2**，我们将使用 `nn.Module` 来定义任意的神经网络架构。
4. 第四部分，PyTorch 序列 API: **抽象层次 3**，我们将使用 `nn.Sequential` 来非常便捷地定义线性前馈网络。
5. 第五部分，CIFAR-10 开放式挑战：请实现你自己的网络，以在 CIFAR-10 上获得尽可能高的准确率。你可以尝试任何层、优化器、超参数或其他高级功能。

以下是对比表格：

API (应用程序接口)	灵活性	便捷性
基础 (Barebone)	高	低
<code>nn.Module</code>	高	中
<code>nn.Sequential</code>	低	高

3 GPU (图形处理器)

你可以在 Colab 上手动切换到 GPU 设备,操作步骤是:点击 Runtime(运行时)-> Change runtime type (更改运行时类型), 然后在 Hardware Accelerator (硬件加速器) 下选择 GPU。你应该在运行以下导入包的单元格之前完成此操作, 因为切换运行时内核会重启。

```
[1]: # 配置 matplotlib 中文字体
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False

import torch # 导入 PyTorch 库
import torch.nn as nn # 导入 PyTorch 的神经网络模块
import torch.optim as optim # 导入 PyTorch 的优化器模块
from torch.utils.data import DataLoader # 导入数据加载器, 用于批量加载数据
from torch.utils.data import sampler # 导入采样器, 用于数据采样
```

```

import torchvision.datasets as dset # 导入 PyTorch 的视觉数据集模块
import torchvision.transforms as T # 导入数据转换模块, 用于数据预处理

import numpy as np # 导入 numpy 库, 用于数值计算

USE_GPU = True # 标记是否使用 GPU
dtype = torch.float32 # 在本教程中我们将全程使用 float 类型

# 如果使用 GPU 且 CUDA 可用, 则将设备设置为 cuda, 否则使用 cpu
if USE_GPU and torch.cuda.is_available():
    device = torch.device('cuda')
else:
    device = torch.device('cpu')

# 控制训练损失打印频率的常量
print_every = 100
print('使用的设备: ', device)

```

使用的设备: cuda

4 第一部分：准备工作

现在, 让我们加载 CIFAR-10 数据集。首次加载可能需要几分钟时间, 但之后文件会被缓存起来。

在本作业的前几部分中, 我们需要自己编写代码来下载 CIFAR-10 数据集、对其进行预处理并以小批量方式迭代; 而 PyTorch 提供了便捷的工具来自动完成这些过程。

[2]: NUM_TRAIN = 49000 # 训练集样本数量

```

# torchvision.transforms 模块提供了数据预处理和数据增强的工具
# 这里我们设置了一个数据转换流程, 通过减去 RGB 通道的均值并除以每个 RGB 通道的标准差来预处理数据
# 这里我们硬编码了均值和标准差 (CIFAR-10 数据集的标准预处理参数)
transform = T.Compose([
    T.ToTensor(), # 将图像转换为 PyTorch 张量 (Tensor)
    T.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010))
    ↪ # 标准化处理
])

```

```

# 我们为每个数据拆分（训练集/验证集/测试集）创建一个 Dataset 对象
# Dataset 每次加载一个训练样本，因此我们将每个 Dataset 包装在 DataLoader 中
# DataLoader 会迭代遍历 Dataset 并形成小批量数据
# 我们通过向 DataLoader 传递一个 Sampler 对象来告诉它如何从底层 Dataset 中采样，
# 从而将 CIFAR-10 训练集拆分为训练集和验证集
cifar10_train = dset.CIFAR10('./cs231n/datasets', train=True, download=True,
                             transform=transform) # 训练集 Dataset, 应用预处理
loader_train = DataLoader(cifar10_train, batch_size=64,
                           sampler=sampler.
↳ SubsetRandomSampler(range(NUM_TRAIN))) # 训练集数据加载器，采样前 49000 个样本

cifar10_val = dset.CIFAR10('./cs231n/datasets', train=True, download=True,
                           transform=transform) # 验证集 Dataset（基于训练集的剩
余部分）
loader_val = DataLoader(cifar10_val, batch_size=64,
                        sampler=sampler.SubsetRandomSampler(range(NUM_TRAIN,
↳ 50000))) # 验证集数据加载器，采样 49000 到 50000 之间的样本

cifar10_test = dset.CIFAR10('./cs231n/datasets', train=False, download=True,
                             transform=transform) # 测试集 Dataset
loader_test = DataLoader(cifar10_test, batch_size=64) # 测试集数据加载器

```

```

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified

```

5 第二部分：基础 PyTorch

PyTorch 附带了高级 API 来帮助我们方便地定义模型架构，这部分内容我们将在本教程的第二部分介绍。在本节中，我们将从最基础的 PyTorch 元素开始，以更好地理解自动求导引擎。完成这个练习后，你会更能体会到高级模型 API 的便捷之处。

我们将从一个简单的全连接 ReLU 网络开始，该网络有两个隐藏层且不带偏置，用于 CIFAR 分类任务。这个实现使用 PyTorch 张量 (Tensors) 上的操作来计算前向传播，并使用 PyTorch 的自动求导功能来计算梯度。你需要理解每一行代码，因为之后你要在此示例基础上编写一个更复杂的版本。

当我们创建一个 `requires_grad=True` 的 PyTorch 张量时，涉及该张量的操作不仅会计算数值，还会在后台构建一个计算图，这使我们能够轻松地通过该图进行反向传播，以计算某些张量相对于下游损失的梯度。具体来说，如果 `x` 是一个 `x.requires_grad == True` 的张量，那么在反向传播之后，`x.grad` 将是另一个张量，其中存储着 `x` 相对于最终标量损失的梯度。

5.0.1 PyTorch 张量：Flatten 函数

PyTorch 张量在概念上类似于 numpy 数组：它是一个 n 维数字网格，并且和 numpy 一样，PyTorch 提供了许多函数来高效地对张量进行操作。作为一个简单的例子，我们在下面提供了一个 `flatten` 函数，该函数对图像数据进行重塑，以便在全连接神经网络中使用。

回想一下，图像数据通常存储在形状为 $N \times C \times H \times W$ 的张量中，其中： N 是数据点的数量 C 是通道的数量 H 是中间特征图的高度（以像素为单位） W 是中间特征图的宽度（以像素为单位）

当我们进行诸如二维卷积之类的操作时，这种数据表示方式是合适的，因为卷积操作需要了解中间特征彼此之间的空间位置关系。然而，当我们使用全连接仿射层处理图像时，我们希望每个数据点由一个单一的向量来表示——将数据的不同通道、行和列分开已经没有意义了。因此，我们使用“flatten”操作将每个表示中的 $C \times H \times W$ 值折叠成一个单一的长向量。下面的 `flatten` 函数首先从给定的一批数据中读取 N 、 C 、 H 和 W 的值，然后返回该数据的一个“视图（view）”。“view”类似于 numpy 中的“reshape”方法：它将 `x` 的维度重塑为 $N \times ??$ ，其中?? 可以是任意值（在这种情况下，它将是 $C \times H \times W$ ，但我们不需要显式地指定）。

```
[3]: def flatten(x):
      N = x.shape[0] # 获取 N (数据点数量), x 的形状为 (N, C, H, W)
      return x.view(N, -1) # 将每个图像的 C*H*W 值"展平"为一个单一向量, -1 表示自动计算该维度大小

      def test_flatten():
          # 创建一个形状为 (2, 1, 3, 2) 的张量 (2 个数据点, 1 个通道, 高 3, 宽 2)
          x = torch.arange(12).view(2, 1, 3, 2)
          print('展平前: ', x)
          print('展平后: ', flatten(x))

      test_flatten() # 调用测试函数
```

```
展平前: tensor([[[[ 0,  1],
                  [ 2,  3],
                  [ 4,  5]]],
```

```

        [[ 6,  7],
         [ 8,  9],
         [10, 11]]]])
展平后: tensor([[ 0,  1,  2,  3,  4,  5],
                [ 6,  7,  8,  9, 10, 11]])

```

5.0.2 基础 PyTorch: 两层网络

这里我们定义一个函数 `two_layer_fc`, 它对一批图像数据执行两层全连接 ReLU 网络的前向传播。定义完前向传播后, 我们通过让全零数据流经网络, 来检查它是否会崩溃以及是否能生成正确形状的输出。

你不需要在这里编写任何代码, 但阅读并理解这个实现很重要。

```
[4]: import torch.nn.functional as F # 导入有用的无状态函数 (如激活函数等)
```

```

def two_layer_fc(x, params):
    """
    一个全连接神经网络; 架构为:
    全连接层 -> ReLU 激活 -> 全连接层。
    注意: 此函数仅定义前向传播;
    PyTorch 会为我们处理反向传播。

    网络的输入将是一批数据, 形状为
    (N, d1, ..., dM), 其中 d1 * ... * dM = D。隐藏层将有 H 个单元,
    输出层将为 C 个类别生成分数。

    输入:
    - x: 形状为 (N, d1, ..., dM) 的 PyTorch 张量, 代表一批输入数据。
    - params: 网络权重的 PyTorch 张量列表 [w1, w2];
      w1 的形状为 (D, H), w2 的形状为 (H, C)。

    返回:
    - scores: 形状为 (N, C) 的 PyTorch 张量, 代表输入数据 x 的分类分数。
    """
    # 首先展平图像
    x = flatten(x) # 形状: [batch_size, C x H x W]

```

```

w1, w2 = params # 从参数列表中获取权重

# 前向传播：使用张量操作计算预测值 y。由于 w1 和
# w2 的 requires_grad=True，涉及这些张量的操作会使
# PyTorch 构建计算图，从而实现梯度的自动计算。
# 因为我们不再手动实现反向传播，所以不需要保留中间值的引用。
# 也可以使用`.clamp(min=0)`，与 F.relu() 等价
x = F.relu(x.mm(w1)) # 第一层全连接 + ReLU 激活
x = x.mm(w2) # 第二层全连接
return x

def two_layer_fc_test():
    hidden_layer_size = 42 # 隐藏层大小
    # 创建输入张量：批大小 64，特征维度 50
    x = torch.zeros((64, 50), dtype=dtype)
    # 初始化权重张量
    w1 = torch.zeros((50, hidden_layer_size), dtype=dtype)
    w2 = torch.zeros((hidden_layer_size, 10), dtype=dtype)
    # 计算网络输出分数
    scores = two_layer_fc(x, [w1, w2])
    print(scores.size()) # 应该输出 [64, 10]

two_layer_fc_test() # 调用测试函数

```

```
torch.Size([64, 10])
```

5.0.3 基础 PyTorch：三层卷积网络

在这里，你需要完成函数 `three_layer_convnet` 的实现，该函数将执行一个三层卷积网络的前向传播。和上面一样，我们可以通过让全零数据流经网络来立即测试实现是否正确。该网络应具有以下架构：

1. 一个带偏置的卷积层，包含 `channel_1` 个滤波器，每个滤波器的形状为 `KW1 x KH1`，零填充为 2
2. ReLU 非线性激活
3. 一个带偏置的卷积层，包含 `channel_2` 个滤波器，每个滤波器的形状为 `KW2 x KH2`，零填充

为 1

4. ReLU 非线性激活
5. 带偏置的全连接层，生成为 C 个类别对应的分数

注意：在全连接层之后**没有** softmax 激活：这是因为 PyTorch 的交叉熵损失会为你执行 softmax 激活，将这一步骤整合在一起可以提高计算效率。

提示：关于卷积操作：参考 <http://pytorch.org/docs/stable/nn.html#torch.nn.functional.conv2d>；注意卷积滤波器的形状！

```
[6]: def three_layer_convnet(x, params):  
    """  
    执行上述架构定义的三层卷积网络的前向传播。  
  
    输入：  
    - x: 形状为  $(N, 3, H, W)$  的 PyTorch 张量，表示一批图像 ( $N$  为批大小，3 为 RGB 通道数)  
    - params: 包含网络权重和偏置的 PyTorch 张量列表；应包含以下内容：  
        - conv_w1: 形状为  $(channel\_1, 3, KH1, KW1)$  的 PyTorch 张量，为第一层卷积层的权重  
            ( $channel\_1$  为输出通道数，3 为输入通道数， $KH1$ 、 $KW1$  为滤波器高和宽)  
        - conv_b1: 形状为  $(channel\_1,)$  的 PyTorch 张量，为第一层卷积层的偏置  
        - conv_w2: 形状为  $(channel\_2, channel\_1, KH2, KW2)$  的 PyTorch 张量，为第二层卷积层的权重  
            ( $channel\_2$  为该层输出通道数， $channel\_1$  为输入通道数)  
        - conv_b2: 形状为  $(channel\_2,)$  的 PyTorch 张量，为第二层卷积层的偏置  
        - fc_w: 全连接层的权重 PyTorch 张量。你能弄清楚它的形状应该是什么吗？  
            (提示：输入维度为  $channel\_2 * H' * W'$ ，输出维度为  $C$ )  
        - fc_b: 全连接层的偏置 PyTorch 张量。你能弄清楚它的形状应该是什么吗？  
            (提示：形状应为  $(C,)$ ， $C$  为类别数)  
  
    返回：  
    - scores: 形状为  $(N, C)$  的 PyTorch 张量，表示  $x$  的分类分数  
    """  
    conv_w1, conv_b1, conv_w2, conv_b2, fc_w, fc_b = params # 从参数列表中解析各层参数  
    scores = None # 初始化分类分数
```



```

    ↪ #####
    # 待办：实现三层卷积网络的前向传播。
    ↪ #

    ↪ #####
    # 第一层卷积 + 填充 2 + ReLU
    x = F.conv2d(x, conv_w1, conv_b1, padding=2)
    x = F.relu(x)

    # 第二层卷积 + 填充 1 + ReLU
    x = F.conv2d(x, conv_w2, conv_b2, padding=1)
    x = F.relu(x)

    # 展平后接全连接层
    x = flatten(x)
    scores = x.mm(fc_w) + fc_b # 全连接层计算分类分数

    ↪ #####
    #                                     你的代码结束
    ↪ #

    ↪ #####
    return scores

```

在定义好上述卷积网络的前向传播后，运行下面的单元格来测试你的实现。

当你运行这个函数时，输出的分数（scores）应该具有（64，10）的形状。

```

[22]: def three_layer_convnet_test():
    # 创建输入张量：批大小 64, 3 个通道 (RGB), 图像尺寸 32x32
    x = torch.zeros((64, 3, 32, 32), dtype=dtype)

    # 第一层卷积层参数：6 个输出通道, 3 个输入通道, 卷积核大小 5x5
    conv_w1 = torch.zeros((6, 3, 5, 5), dtype=dtype) # [输出通道数, 输入通道数,
    ↪ 卷积核高度, 卷积核宽度]
    conv_b1 = torch.zeros((6,)) # 输出通道数对应的偏置

```

```

# 第二层卷积层参数: 9 个输出通道, 6 个输入通道 (与上层输出通道数一致), 卷积核大小 3x3
conv_w2 = torch.zeros((9, 6, 3, 3), dtype=dtype) # [输出通道数, 输入通道数, 卷积核高度, 卷积核宽度]
conv_b2 = torch.zeros((9,)) # 输出通道数对应的偏置

# 全连接层参数: 需要根据两层卷积后的张量形状计算输入维度
# 此处假设两层卷积后张量形状为 [9, 32, 32] (实际需根据卷积参数重新计算)
fc_w = torch.zeros((9 * 32 * 32, 10)) # 输入维度为卷积输出的展平尺寸, 输出维度为 10 (类别数)
fc_b = torch.zeros(10) # 10 个类别的偏置

# 计算三层卷积网络的输出分数
scores = three_layer_convnet(x, [conv_w1, conv_b1, conv_w2, conv_b2, fc_w, fc_b])
print(scores.size()) # 应该输出 [64, 10] (64 个样本, 10 个类别分数)

# 调用测试函数
three_layer_convnet_test()

```

```
torch.Size([64, 10])
```

5.0.4 基础 PyTorch: 初始化

让我们编写几个工具方法来初始化模型的权重矩阵。

- `random_weight(shape)` 使用 Kaiming 归一化方法初始化权重张量。
- `zero_weight(shape)` 初始化全零权重张量。适用于实例化偏置参数。

`random_weight` 函数使用 Kaiming 正态初始化方法, 相关描述见于:

He et al, *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*, ICCV 2015, <https://arxiv.org/abs/1502.01852>

```
[8]: def random_weight(shape):
    """
    为权重创建随机张量; 设置 requires_grad=True 意味着我们
    希望在反向传播过程中为这些张量计算梯度。
    """

```

```

我们使用 Kaiming 归一化:  $\text{sqrt}(2 / \text{fan\_in})$ 
"""
if len(shape) == 2: # 全连接层权重
    fan_in = shape[0] # 输入特征数
else:
    # 卷积层权重 [输出通道数, 输入通道数, 核高, 核宽]
    fan_in = np.prod(shape[1:]) # 输入特征数 (计算输入通道数与核尺寸的乘积)
# randn 生成标准正态分布的随机数
w = torch.randn(shape, device=device, dtype=dtype) * np.sqrt(2. / fan_in)
w.requires_grad = True # 启用梯度计算
return w

def zero_weight(shape):
    # 创建全零张量, 启用梯度计算
    return torch.zeros(shape, device=device, dtype=dtype, requires_grad=True)

# 创建一个形状为 [3 x 5] 的权重
# 如果你使用 GPU, 应该看到类型为 `torch.cuda.FloatTensor`
# 否则应该是 `torch.FloatTensor`
random_weight((3, 5))

```

```

[8]: tensor([[ 0.2400, -0.0116, -0.5420,  0.8472,  0.4956],
            [-0.7429,  0.9353, -1.3903, -1.4174,  0.5391],
            [-0.6296, -0.6637,  0.3131,  0.4623,  0.3218]], device='cuda:0',
        requires_grad=True)

```

5.0.5 基础 PyTorch: 检查准确率

在训练模型时, 我们会使用以下函数来检查模型在训练集或验证集上的准确率。

检查准确率时, 我们不需要计算任何梯度; 因此, 在计算分数时, 我们不需要 PyTorch 为我们构建计算图。为了阻止图的构建, 我们将计算放在 `torch.no_grad()` 上下文管理器的作用域内。

```

[14]: def check_accuracy_part2(loader, model_fn, params):
        """
        检查分类模型的准确率。

        输入:

```

- *loader*: 我们要检查的数据分割对应的 *DataLoader*
- *model_fn*: 执行模型前向传播的函数, 签名为 `scores = model_fn(x, params)`
- *params*: 给出模型参数的 *PyTorch* 张量列表

返回: 无返回值, 但会打印模型的准确率

```
"""
# 判断是验证集还是测试集 (根据数据集的 train 属性)
split = 'val' if loader.dataset.train else 'test'
print('正在检查%s集上的准确率' % split)
num_correct, num_samples = 0, 0 # 正确预测数和总样本数
# 禁用梯度计算 (检查准确率时不需要)
with torch.no_grad():
    for x, y in loader:
        # 将输入数据移动到指定设备 (如 GPU) 并转换为指定数据类型
        x = x.to(device=device, dtype=dtype)
        # 将标签移动到指定设备并转换为长整数类型
        y = y.to(device=device, dtype=torch.int64)
        # 计算模型输出分数
        scores = model_fn(x, params)
        # 找到分数最高的类别作为预测结果
        _, preds = scores.max(1)
        # 累加正确预测的数量和总样本数量
        num_correct += (preds == y).sum()
        num_samples += preds.size(0)
# 计算准确率
acc = float(num_correct) / num_samples
print('正确数: %d / 总数: %d (准确率: %.2f%%)' % (num_correct,
↪num_samples, 100 * acc))
```

5.0.6 基础 PyTorch: 训练循环

我们现在可以搭建一个基本的训练循环来训练我们的网络。我们将使用不带动量的随机梯度下降来训练模型。我们会使用 `torch.functional.cross_entropy` 来计算损失; 你可以在[这里查看相关说明](#)。

训练循环接收神经网络函数、初始化参数列表 (在我们的示例中是 `[w1, w2]`) 和学习率作为输入。

```
[41]: def train_part2(model_fn, params, learning_rate):
    """
    在 CIFAR-10 数据集上训练模型。

    输入:
    - model_fn: 执行模型前向传播的 Python 函数。
      函数签名应为 scores = model_fn(x, params), 其中 x 是图像数据的 PyTorch 张量,
      params 是给出模型权重的 PyTorch 张量列表, scores 是形状为  $(N, C)$  的 PyTorch 张量,
      给出 x 中元素的分类分数。
    - params: 给出模型权重的 PyTorch 张量列表
    - learning_rate: 用于 SGD 的学习率 (Python 标量)

    返回: 无
    """
    for t, (x, y) in enumerate(loader_train):
        # 将数据移动到合适的设备 (GPU 或 CPU)
        x = x.to(device=device, dtype=dtype)
        y = y.to(device=device, dtype=torch.long)

        # 前向传播: 计算分数和损失
        scores = model_fn(x, params)
        loss = F.cross_entropy(scores, y)

        # 反向传播: PyTorch 会找出计算图中所有 requires_grad=True 的张量,
        # 使用反向传播计算损失相对于这些张量的梯度, 并将梯度存储在每个张量的 .grad
        # 属性中。
        loss.backward()

        # 更新参数。我们不希望通过参数更新进行反向传播,
        # 因此将更新操作放在 torch.no_grad() 上下文管理器中, 以防止构建计算图。
        with torch.no_grad():
            for w in params:
                # 有些参数在反向传播中可能不会收到梯度 (例如未被使用的参数),
                # 此时 w.grad 为 None, 直接参与数值运算会报错, 因此跳过这些参数。
```

```

        if w.grad is None:
            continue

        w -= learning_rate * w.grad # 参数更新公式

        # 在反向传播后手动清零梯度
        w.grad.zero_()

    # 定期打印训练信息和验证集准确率
    if t % print_every == 0:
        print('迭代次数 %d, 损失 = %.4f' % (t, loss.item()))
        check_accuracy_part2(loader_val, model_fn, params)
        print()

```

5.0.7 基础 PyTorch: 训练一个两层网络

现在我们可以运行训练循环了。我们需要为全连接权重 w_1 和 w_2 显式分配张量。

CIFAR 的每个小批量有 64 个样本，因此张量形状为 $[64, 3, 32, 32]$ 。

展平后， x 的形状应为 $[64, 3 * 32 * 32]$ 。这将是 w_1 第一维度的大小。 w_1 的第二维度是隐藏层大小，它也将是 w_2 的第一维度。

最后，网络的输出是一个 10 维向量，代表 10 个类别上的概率分布。

你不需要调整任何超参数，但训练一个轮次后，你应该能看到准确率超过 40%。

```

[16]: hidden_layer_size = 4000 # 隐藏层大小
      learning_rate = 1e-2 # 学习率

      # 初始化第一层权重: 输入维度为 3*32*32 (展平后的图像), 输出维度为隐藏层大小
      w1 = random_weight((3 * 32 * 32, hidden_layer_size))
      # 初始化第二层权重: 输入维度为隐藏层大小, 输出维度为 10 (CIFAR-10 的类别数)
      w2 = random_weight((hidden_layer_size, 10))

      # 训练两层全连接网络
      train_part2(two_layer_fc, [w1, w2], learning_rate)

```

迭代次数 0, 损失 = 2.8041

正在检查 val 集上的准确率

正确数: 134 / 总数: 1000 (准确率: 13.40%)

迭代次数 100, 损失 = 2.7315

正在检查 val 集上的准确率

正确数: 290 / 总数: 1000 (准确率: 29.00%)

迭代次数 200, 损失 = 2.1077

正在检查 val 集上的准确率

正确数: 337 / 总数: 1000 (准确率: 33.70%)

迭代次数 300, 损失 = 1.7597

正在检查 val 集上的准确率

正确数: 414 / 总数: 1000 (准确率: 41.40%)

迭代次数 400, 损失 = 1.8172

正在检查 val 集上的准确率

正确数: 437 / 总数: 1000 (准确率: 43.70%)

迭代次数 500, 损失 = 1.6777

正在检查 val 集上的准确率

正确数: 436 / 总数: 1000 (准确率: 43.60%)

迭代次数 600, 损失 = 1.8295

正在检查 val 集上的准确率

正确数: 403 / 总数: 1000 (准确率: 40.30%)

迭代次数 700, 损失 = 1.7755

正在检查 val 集上的准确率

正确数: 408 / 总数: 1000 (准确率: 40.80%)

5.0.8 基础 PyTorch: 训练一个卷积网络

在下面的内容中, 你需要使用上面定义的函数在 CIFAR 数据集上训练一个三层卷积网络。该网络应具有以下架构:

1. 带偏置的卷积层, 包含 32 个 5×5 的滤波器, 零填充为 2
2. ReLU 激活函数

3. 带偏置的卷积层，包含 16 个 3×3 的滤波器，零填充为 1
4. ReLU 激活函数
5. 带偏置的全连接层，用于计算 10 个类别的分数

你应该使用上面定义的 `random_weight` 函数初始化权重矩阵，使用上面的 `zero_weight` 函数初始化偏置向量。

你不需要调整任何超参数，但如果一切正常，训练一个轮次后你应该能达到 42% 以上的准确率。

```
[42]: learning_rate = 3e-3 # 学习率

channel_1 = 32 # 第一层卷积输出通道数
channel_2 = 16 # 第二层卷积输出通道数

# 初始化各层参数 (初始化为 None)
conv_w1 = None # 第一层卷积权重
conv_b1 = None # 第一层卷积偏置
conv_w2 = None # 第二层卷积权重
conv_b2 = None # 第二层卷积偏置
fc_w = None # 全连接层权重
fc_b = None # 全连接层偏置

#####
# 待办：初始化三层卷积网络的参数。 #
#####

# 第一层卷积：32 个 5x5 滤波器，输入通道 3，零填充 2
conv_w1 = random_weight((channel_1, 3, 5, 5))
conv_b1 = zero_weight((channel_1,))

# 第二层卷积：16 个 3x3 滤波器，输入通道 32，零填充 1
conv_w2 = random_weight((channel_2, channel_1, 3, 3))
conv_b2 = zero_weight((channel_2,))

# 全连接层：输入维度为 channel_2 * 32 * 32 (卷积后特征图尺寸不变)，输出 10 个类别
fc_w = random_weight((channel_2 * 32 * 32, 10))
fc_b = zero_weight((10,))
```



```
#####
#                                     你的代码结束
#
#####

# 参数列表
params = [conv_w1, conv_b1, conv_w2, conv_b2, fc_w, fc_b]
# 训练三层卷积网络
train_part2(three_layer_convnet, params, learning_rate)
```

迭代次数 0, 损失 = 2.9248

正在检查 val 集上的准确率

正确数: 112 / 总数: 1000 (准确率: 11.20%)

迭代次数 100, 损失 = 1.8694

正在检查 val 集上的准确率

正确数: 329 / 总数: 1000 (准确率: 32.90%)

迭代次数 200, 损失 = 1.6602

正在检查 val 集上的准确率

正确数: 400 / 总数: 1000 (准确率: 40.00%)

迭代次数 300, 损失 = 1.6690

正在检查 val 集上的准确率

正确数: 426 / 总数: 1000 (准确率: 42.60%)

迭代次数 400, 损失 = 1.5666

正在检查 val 集上的准确率

正确数: 458 / 总数: 1000 (准确率: 45.80%)

迭代次数 500, 损失 = 1.4914

正在检查 val 集上的准确率

正确数: 465 / 总数: 1000 (准确率: 46.50%)

迭代次数 600, 损失 = 1.4380

正在检查 val 集上的准确率

正确数: 457 / 总数: 1000 (准确率: 45.70%)

迭代次数 700, 损失 = 1.3421
正在检查 val 集上的准确率
正确数: 479 / 总数: 1000 (准确率: 47.90%)

6 第三部分. PyTorch 模块 API

基础的 PyTorch 要求我们手动跟踪所有的参数张量。对于只有几个张量的小型网络来说这没什么问题，但在更大的网络中跟踪数十或数百个张量会极其不便且容易出错。

PyTorch 提供了 `nn.Module` API, 让你可以定义任意的网络架构，同时自动为你跟踪所有可学习的参数。在第二部分中，我们自己实现了 SGD。PyTorch 还提供了 `torch.optim` 包，其中实现了所有常见的优化器，如 RMSProp、Adagrad 和 Adam。它甚至支持近似的二阶方法，如 L-BFGS！你可以参考[文档](#)了解每个优化器的确切规格。

要使用 Module API，请遵循以下步骤：

1. 继承 `nn.Module`。给你的网络类起一个直观的名字，比如 `TwoLayerFC`。
2. 在构造函数 `__init__()` 中，将所有需要的层定义为类属性。像 `nn.Linear` 和 `nn.Conv2d` 这样的层对象本身就是 `nn.Module` 的子类，并且包含可学习的参数，因此你不必自己实例化原始张量。`nn.Module` 会为你跟踪这些内部参数。参考[文档](#)了解数十种内置层的更多信息。**警告：**不要忘记先调用 `super().__init__()`！
3. 在 `forward()` 方法中，定义网络的连接方式。你应该将在 `__init__` 中定义的属性用作函数调用，接收张量作为输入并输出“转换后的”张量。不要在 `forward()` 中创建任何带有可学习参数的新层！所有这些层都必须在 `__init__` 中预先声明。

定义好 Module 子类后，你可以将其实例化为一个对象，并像第二部分中的神经网络前向函数一样调用它。

6.0.1 模块 API: 两层网络

下面是一个两层全连接网络的具体示例：

```
[43]: class TwoLayerFC(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super().__init__() # 调用父类 nn.Module 的构造函数
        # 将层对象分配为类属性
        self.fc1 = nn.Linear(input_size, hidden_size) # 第一层全连接层
```

```

# nn.init 包包含便捷的初始化方法
# http://pytorch.org/docs/master/nn.html#torch-nn-init
nn.init.kaiming_normal_(self.fc1.weight) # 使用 Kaiming 正态分布初始化第
一层权重

self.fc2 = nn.Linear(hidden_size, num_classes) # 第二层全连接层
nn.init.kaiming_normal_(self.fc2.weight) # 使用 Kaiming 正态分布初始化第
二层权重

def forward(self, x):
    # forward 方法始终定义网络的连接方式
    x = flatten(x) # 展平输入张量
    # 前向传播: 第一层 -> ReLU 激活 -> 第二层
    scores = self.fc2(F.relu(self.fc1(x)))
    return scores

def test_TwoLayerFC():
    input_size = 50 # 输入特征维度
    # 创建输入张量: 批大小 64, 特征维度 50
    x = torch.zeros((64, input_size), dtype=dtype)
    # 实例化两层全连接网络: 输入维度 50, 隐藏层大小 42, 输出类别数 10
    model = TwoLayerFC(input_size, 42, 10)
    scores = model(x) # 计算输出分数
    print(scores.size()) # 应该输出 [64, 10]
test_TwoLayerFC() # 调用测试函数

```

```
torch.Size([64, 10])
```

6.0.2 模块 API: 三层卷积网络

现在轮到你实现一个三层卷积网络，其后跟随一个全连接层。网络架构应与第二部分相同：

1. 包含 `channel_1` 个 5×5 滤波器的卷积层，零填充为 2
2. ReLU 激活函数
3. 包含 `channel_2` 个 3×3 滤波器的卷积层，零填充为 1
4. ReLU 激活函数
5. 输出到 `num_classes` 个类别的全连接层

你应该使用 Kaiming 正态初始化方法初始化模型的权重矩阵。

提示： <http://pytorch.org/docs/stable/nn.html#conv2d>

实现完三层卷积网络后，`test_ThreeLayerConvNet` 函数将运行你的实现；它应打印输出分数的形状为 (64, 10)。

```
[45]: class ThreeLayerConvNet(nn.Module):
    def __init__(self, in_channel, channel_1, channel_2, num_classes):
        super().__init__()

        #####
        # 待办：按照上面定义的架构，设置三层卷积网络所需的层。 #
        #####
        # 第一层：带偏置的卷积层，channel_1 个 5x5 滤波器，padding=2
        self.conv1 = nn.Conv2d(in_channel, channel_1, kernel_size=5, padding=2,
        ↪ bias=True)
        # 第二层：带偏置的卷积层，channel_2 个 3x3 滤波器，padding=1
        self.conv2 = nn.Conv2d(channel_1, channel_2, kernel_size=3, padding=1,
        ↪ bias=True)
        # 全连接层：展平后映射到 num_classes
        self.fc = nn.Linear(channel_2 * 32 * 32, num_classes)

        # 权重初始化：Kaiming 正态，偏置置零
        nn.init.kaiming_normal_(self.conv1.weight)
        nn.init.zeros_(self.conv1.bias)
        nn.init.kaiming_normal_(self.conv2.weight)
        nn.init.zeros_(self.conv2.bias)
        nn.init.kaiming_normal_(self.fc.weight)
        nn.init.zeros_(self.fc.bias)

        #####
        #                                你的代码结束                                ↪
        ↪ #

        #####

    def forward(self, x):
        scores = None

        #####
        # 待办：实现三层卷积网络的前向传播函数。你应该使用在 __init__ 中定义的层， ↪
        ↪ #
```

```

# 并在 forward() 中指定这些层的连接方式
↪ #
#####
x = F.relu(self.conv1(x))
x = F.relu(self.conv2(x))
x = flatten(x)
scores = self.fc(x)
#####
#                                你的代码结束
↪ #
#####
return scores

def test_ThreeLayerConvNet():
    # 创建输入张量: 批大小 64, 图像尺寸 [3, 32, 32] (3 通道, 32x32 像素)
    x = torch.zeros((64, 3, 32, 32), dtype=dtype)
    # 实例化三层卷积网络: 输入通道 3, 第一层输出通道 12, 第二层输出通道 8, 类别数 10
    model = ThreeLayerConvNet(in_channel=3, channel_1=12, channel_2=8,
↪ num_classes=10)
    scores = model(x) # 计算输出分数
    print(scores.size()) # 应该输出 [64, 10]

# 运行测试
test_ThreeLayerConvNet() # 调用测试函数

```

```
torch.Size([64, 10])
```

6.0.3 模块 API: 检查准确率

给定验证集或测试集，我们可以检查神经网络的分类准确率。

这个版本与第二部分中的版本略有不同。你不再需要手动传入参数。

```

[46]: def check_accuracy_part34(loader, model):
        # 判断是验证集还是测试集
        if loader.dataset.train:
            print('正在验证集上检查准确率')

```

```

else:
    print('正在测试集上检查准确率')
    num_correct = 0 # 正确预测的数量
    num_samples = 0 # 总样本数量
    model.eval() # 将模型设置为评估模式 (关闭 dropout 等训练时特有的层)
    # 禁用梯度计算
    with torch.no_grad():
        for x, y in loader:
            # 将输入数据移动到指定设备 (如 GPU) 并转换为指定数据类型
            x = x.to(device=device, dtype=dtype)
            # 将标签移动到指定设备并转换为长整数类型
            y = y.to(device=device, dtype=torch.long)
            # 计算模型输出分数
            scores = model(x)
            # 找到分数最高的类别作为预测结果
            _, preds = scores.max(1)
            # 累加正确预测的数量和总样本数量
            num_correct += (preds == y).sum()
            num_samples += preds.size(0)

        # 计算准确率
        acc = float(num_correct) / num_samples
        print('正确数: %d / 总数: %d (准确率: %.2f%%)' % (num_correct,
↪num_samples, 100 * acc))

```

6.0.4 模块 API: 训练循环

我们还使用了一个略有不同的训练循环。我们不再自己更新权重值，而是使用 `torch.optim` 包中的 `Optimizer` (优化器) 对象，它封装了优化算法的概念，并提供了大多数常用于优化神经网络的算法实现。

```

[47]: def train_part34(model, optimizer, epochs=1):
    """
    使用 PyTorch 的 Module API 在 CIFAR-10 上训练模型。

    输入:
    - model: 要训练的 PyTorch Module 模型

```

- *optimizer*: 用于训练模型的 *Optimizer* 对象
- *epochs*: (可选) 训练的轮次 (*Python* 整数)

返回: 无返回值, 但会在训练过程中打印模型的准确率。

"""

```

model = model.to(device=device) # 将模型参数移动到 CPU/GPU
for e in range(epochs):
    for t, (x, y) in enumerate(loader_train):
        model.train() # 将模型设置为训练模式
        x = x.to(device=device, dtype=dtype) # 移动到设备 (如 GPU) 并转换数
据类型
        y = y.to(device=device, dtype=torch.long) # 移动标签到设备并转换为长
整数类型

        scores = model(x) # 前向传播计算分数
        loss = F.cross_entropy(scores, y) # 计算交叉熵损失

        # 将优化器要更新的所有变量的梯度清零
        optimizer.zero_grad()

        # 反向传播: 计算损失相对于模型每个参数的梯度
        loss.backward()

        # 使用反向传播计算出的梯度实际更新模型参数
        optimizer.step()

        # 定期打印训练信息和验证集准确率
        if t % print_every == 0:
            print('迭代次数 %d, 损失 = %.4f' % (t, loss.item()))
            check_accuracy_part34(loader_val, model)
            print()

```

6.0.5 模块 API: 训练一个两层网络

现在我们可以运行训练循环了。与第二部分不同, 我们不再需要显式地分配参数张量。

只需将输入大小、隐藏层大小和类别数 (即输出大小) 传递给 `TwoLayerFC` 的构造函数即可。

你还需要定义一个优化器，用于跟踪 TwoLayerFC 中所有可学习的参数。

你不需要调整任何超参数，但训练一个轮次后，模型准确率应该能达到 40% 以上。

```
[48]: hidden_layer_size = 4000 # 隐藏层大小
      learning_rate = 1e-2 # 学习率

      # 实例化两层全连接网络：输入大小为 3*32*32 (展平后的图像)，隐藏层大小为 4000，输出类别数为 10
      model = TwoLayerFC(3 * 32 * 32, hidden_layer_size, 10)

      # 定义优化器：使用 SGD 优化器，跟踪模型中的所有参数，学习率为上面定义的值
      optimizer = optim.SGD(model.parameters(), lr=learning_rate)

      # 训练模型
      train_part34(model, optimizer)
```

迭代次数 0, 损失 = 3.6049

正在验证集上检查准确率

正确数: 153 / 总数: 1000 (准确率: 15.30%)

迭代次数 100, 损失 = 2.5059

正在验证集上检查准确率

正确数: 326 / 总数: 1000 (准确率: 32.60%)

迭代次数 200, 损失 = 2.6805

正在验证集上检查准确率

正确数: 359 / 总数: 1000 (准确率: 35.90%)

迭代次数 300, 损失 = 2.5975

正在验证集上检查准确率

正确数: 368 / 总数: 1000 (准确率: 36.80%)

迭代次数 400, 损失 = 2.1595

正在验证集上检查准确率

正确数: 422 / 总数: 1000 (准确率: 42.20%)

迭代次数 500, 损失 = 1.5643

正在验证集上检查准确率

正确数: 422 / 总数: 1000 (准确率: 42.20%)

迭代次数 600, 损失 = 1.6220

正在验证集上检查准确率

正确数: 458 / 总数: 1000 (准确率: 45.80%)

迭代次数 700, 损失 = 1.5735

正在验证集上检查准确率

正确数: 443 / 总数: 1000 (准确率: 44.30%)

6.0.6 模块 API: 训练一个三层卷积网络

现在你应该使用模块 API 在 CIFAR 数据集上训练一个三层卷积网络。这应该和训练两层网络非常相似! 你不需要调整任何超参数, 但训练一个轮次后, 准确率应该能达到 45% 以上。

你应该使用不带动量的随机梯度下降来训练这个模型。

```
[ ]: learning_rate = 3e-3 # 学习率
channel_1 = 32 # 第一层卷积输出通道数
channel_2 = 16 # 第二层卷积输出通道数

model = None # 三层卷积网络模型实例
optimizer = None # 优化器实例

#####
# 待办: 实例化你的 ThreeLayerConvNet 模型和对应的优化器
#
#####

#####
#
# 你的代码结束
#
#####

# 训练模型
train_part34(model, optimizer)
```

7 第四部分. PyTorch 顺序 API

第三部分介绍了 PyTorch 的 Module API，它允许你定义任意的可学习层及其连接方式。

对于像堆叠的前馈层这样的简单模型，你仍然需要经过三个步骤：继承 `nn.Module`、在 `__init__` 中将层分配为类属性、在 `forward()` 中逐个调用每个层。有没有更便捷的方法呢？

幸运的是，PyTorch 提供了一个名为 `nn.Sequential` 的容器模块，它将上述步骤合并为一个。它不像 `nn.Module` 那样灵活，因为你不能指定比前馈堆叠更复杂的拓扑结构，但对于许多使用场景来说已经足够了。

7.0.1 顺序 API: 两层网络

让我们看看如何使用 `nn.Sequential` 重写我们的两层全连接网络示例，并使用上面定义的训练循环来训练它。

同样，这里你不需要调整任何超参数，但训练一个轮次后，准确率应该能达到 40% 以上。

[49]: # 我们需要将 `flatten` 函数包装在一个模块中，以便在 `nn.Sequential` 中堆叠使用

```
class Flatten(nn.Module):
    def forward(self, x):
        return flatten(x) # 展平输入张量

hidden_layer_size = 4000 # 隐藏层大小
learning_rate = 1e-2 # 学习率

# 使用 nn.Sequential 构建两层全连接网络
model = nn.Sequential(
    Flatten(), # 展平操作
    nn.Linear(3 * 32 * 32, hidden_layer_size), # 第一层全连接层
    nn.ReLU(), # ReLU 激活函数
    nn.Linear(hidden_layer_size, 10), # 第二层全连接层 (输出层)
)

# 可以在 optim.SGD 中使用 Nesterov 动量
optimizer = optim.SGD(model.parameters(), lr=learning_rate,
                       momentum=0.9, nesterov=True)

# 训练模型
train_part34(model, optimizer)
```

迭代次数 0, 损失 = 2.2993
正在验证集上检查准确率
正确数: 197 / 总数: 1000 (准确率: 19.70%)

迭代次数 100, 损失 = 1.9978
正在验证集上检查准确率
正确数: 398 / 总数: 1000 (准确率: 39.80%)

迭代次数 200, 损失 = 1.8365
正在验证集上检查准确率
正确数: 414 / 总数: 1000 (准确率: 41.40%)

迭代次数 300, 损失 = 1.4528
正在验证集上检查准确率
正确数: 413 / 总数: 1000 (准确率: 41.30%)

迭代次数 400, 损失 = 1.6590
正在验证集上检查准确率
正确数: 455 / 总数: 1000 (准确率: 45.50%)

迭代次数 500, 损失 = 1.6977
正在验证集上检查准确率
正确数: 408 / 总数: 1000 (准确率: 40.80%)

迭代次数 600, 损失 = 1.6986
正在验证集上检查准确率
正确数: 423 / 总数: 1000 (准确率: 42.30%)

迭代次数 700, 损失 = 2.0510
正在验证集上检查准确率
正确数: 411 / 总数: 1000 (准确率: 41.10%)

7.0.2 顺序 API: 三层卷积网络

在这里, 你需要使用 `nn.Sequential` 来定义和训练一个三层卷积网络, 其架构与我们在第三部分中使用的相同:

1. 带偏置的卷积层，包含 32 个 5x5 的滤波器，零填充为 2
2. ReLU 激活函数
3. 带偏置的卷积层，包含 16 个 3x3 的滤波器，零填充为 1
4. ReLU 激活函数
5. 带偏置的全连接层，用于计算 10 个类别的分数

你可以使用 PyTorch 默认的权重初始化方式。

你应该使用带有 Nesterov 动量（动量值为 0.9）的随机梯度下降来优化模型。

同样，你不需要调整任何超参数，但训练一个轮次后，准确率应该能达到 55% 以上。

```
[63]: channel_1 = 32  # 第一层卷积输出通道数
channel_2 = 16  # 第二层卷积输出通道数
learning_rate = 1e-2  # 学习率

model = None  # 三层卷积网络模型（使用 Sequential API 构建）
optimizer = None  # 优化器

#####
# 待办：使用 Sequential API 重写第三部分中带偏置的两层卷积网络。
#
# （注：原文"2-layer ConvNet" 应为笔误，结合上下文应为三层卷积网络）
#####
# 使用模块 API 实例化三层卷积网络并移动到指定设备
model = ThreeLayerConvNet(in_channel=3, channel_1=channel_1,
    channel_2=channel_2, num_classes=10)
model = model.to(device=device)

# 使用带 Nesterov 动量的 SGD 优化器
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate, momentum=0.9,
    nesterov=True)

#####
#                                     你的代码结束                                     #
#####

# 训练模型
train_part34(model, optimizer)
```

迭代次数 0, 损失 = 2.9369

正在验证集上检查准确率

正确数: 90 / 总数: 1000 (准确率: 9.00%)

迭代次数 100, 损失 = 1.7456

正在验证集上检查准确率

正确数: 414 / 总数: 1000 (准确率: 41.40%)

迭代次数 200, 损失 = 1.3571

正在验证集上检查准确率

正确数: 487 / 总数: 1000 (准确率: 48.70%)

迭代次数 300, 损失 = 1.5494

正在验证集上检查准确率

正确数: 515 / 总数: 1000 (准确率: 51.50%)

迭代次数 400, 损失 = 1.3732

正在验证集上检查准确率

正确数: 508 / 总数: 1000 (准确率: 50.80%)

迭代次数 500, 损失 = 1.2238

正在验证集上检查准确率

正确数: 537 / 总数: 1000 (准确率: 53.70%)

迭代次数 600, 损失 = 1.2774

正在验证集上检查准确率

正确数: 555 / 总数: 1000 (准确率: 55.50%)

迭代次数 700, 损失 = 1.3721

正在验证集上检查准确率

正确数: 557 / 总数: 1000 (准确率: 55.70%)

8 第五部分. CIFAR-10 开放式挑战

在本部分中, 你可以在 CIFAR-10 数据集上尝试任何你喜欢的卷积网络架构。

现在，你的任务是通过尝试不同的架构、超参数、损失函数和优化器来训练模型，使其在 10 个轮次内，在 CIFAR-10 **验证集**上达到**至少 70%** 的准确率。你可以使用上面的 `check_accuracy` 函数和 `train` 函数。你可以使用 `nn.Module` 或 `nn.Sequential` API。

在本笔记本的末尾描述你所做的工作。

以下是每个组件的官方 API 文档。注意：我们在课堂上所说的“空间批归一化 (spatial batch norm)”在 PyTorch 中称为“BatchNorm2D”。

- torch.nn 包中的层：<http://pytorch.org/docs/stable/nn.html>
- 激活函数：<http://pytorch.org/docs/stable/nn.html#non-linear-activations>
- 损失函数：<http://pytorch.org/docs/stable/nn.html#loss-functions>
- 优化器：<http://pytorch.org/docs/stable/optim.html>

8.0.1 你可以尝试的方向：

- **滤波器大小**：上面我们使用了 5x5 的滤波器；更小的滤波器会更高效吗？
- **滤波器数量**：上面我们使用了 32 个滤波器。更多或更少的滤波器会表现更好吗？
- **池化与步长卷积**：你会使用最大池化还是步长卷积？
- **批归一化**：尝试在卷积层后添加空间批归一化，在仿射层后添加普通批归一化。你的网络训练速度会更快吗？
- **网络架构**：上面的网络有两层可训练参数。更深的网络会表现更好吗？值得尝试的良好架构包括：
 - [卷积-激活-池化] $\times N \rightarrow$ [仿射] $\times M \rightarrow$ [softmax 或 SVM]
 - [卷积-激活-卷积-激活-池化] $\times N \rightarrow$ [仿射] $\times M \rightarrow$ [softmax 或 SVM]
 - [批归一化-激活-卷积] $\times N \rightarrow$ [仿射] $\times M \rightarrow$ [softmax 或 SVM]
- **全局平均池化**：不是先展平然后使用多个仿射层，而是执行卷积直到图像变得很小 (大约 7x7)，然后执行平均池化操作得到 1x1 的图像 (1, 1, 滤波器数量)，再将其重塑为 (滤波器数量) 维的向量。这在谷歌的 Inception 网络中使用 (参见表 1 了解其架构)。
- **正则化**：添加 l2 权重正则化，或者可能使用 Dropout。

8.0.2 训练技巧

对于你尝试的每个网络架构，你都应该调整学习率和其他超参数。在这过程中，有几个重要的事情需要记住：

- 如果参数设置合理，你应该在几百次迭代内看到性能提升
- 记住超参数调优的“由粗到细”方法：先在较大的超参数范围内测试，只训练几次迭代，找出那些有效的参数组合。
- 一旦你找到了一些似乎有效的参数集，在这些参数周围进行更精细的搜索。你可能需要训练更多的轮次。

- 你应该使用验证集进行超参数搜索，并保留测试集用于在由验证集选出的最佳参数上评估你的架构。

8.0.3 更进一步

如果你有兴趣，还有许多其他功能可以实现来尝试提高性能。你**不需要**实现这些，但如果有时间，不妨尝试一下，会很有趣！

- 替代优化器：你可以尝试 Adam、Adagrad、RMSprop 等。
- 替代激活函数，如 leaky ReLU、参数化 ReLU、ELU 或 MaxOut。
- 模型集成
- 数据增强
- 新架构
 - ResNets，其中前一层的输入被添加到输出中。
 - DenseNets，其中前几层的输入被连接在一起。
 - [这个博客有深入的概述](#)

8.0.4 祝你训练愉快！

```
[64]: #####
# 待办：
#
# 尝试各种架构、优化器和超参数。
# 在 10 个轮次内，在 * 验证集 * 上达到至少 70% 的准确率。
#
#
# 注意，你可以使用 check_accuracy 函数来评估测试集或验证集，通过将 loader_test 或
# loader_val 作为第二个参数传递给 check_accuracy。在完成架构和超参数调整之前，
#
# 不应触碰测试集，只在最后运行一次测试集以报告最终结果。
#####
# 这里构建一个更强的卷积网络：
# - 使用多个卷积块（包含 BatchNorm + ReLU），中间使用 MaxPool2d 下采样。
# - 两次下采样后，特征图尺寸从 32x32 -> 16x16 -> 8x8，便于接入全连接层。
import math

model = nn.Sequential(
```

```

nn.Conv2d(3, 64, kernel_size=5, padding=2, bias=True),
nn.ReLU(),
nn.BatchNorm2d(64),

nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=True),
nn.ReLU(),
nn.MaxPool2d(2),

nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=True),
nn.ReLU(),
nn.BatchNorm2d(128),
nn.MaxPool2d(2),

nn.Flatten(),
nn.Linear(128 * 8 * 8, 512, bias=True),
nn.ReLU(),
nn.Linear(512, 10, bias=True)
)

# 权重初始化 (Kaiming 正态) 和偏置归零
for m in model.modules():
    if isinstance(m, nn.Conv2d) or isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight)
        if m.bias is not None:
            nn.init.zeros_(m.bias)

# 将模型移动到设备
model = model.to(device=device)

# 优化器: SGD + 动量 + Nesterov + 小的权重衰减
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate, momentum=0.9,
    ↪nesterov=True, weight_decay=5e-4)

#####
#                                     你的代码结束                                     #
#####

```



```
# 你的模型应该至少达到 70% 的准确率
```

```
train_part34(model, optimizer, epochs=10) # 训练模型, 共 10 个轮次
```

迭代次数 0, 损失 = 3.8296

正在验证集上检查准确率

正确数: 155 / 总数: 1000 (准确率: 15.50%)

迭代次数 100, 损失 = 1.6389

正在验证集上检查准确率

正确数: 460 / 总数: 1000 (准确率: 46.00%)

迭代次数 200, 损失 = 1.3307

正在验证集上检查准确率

正确数: 529 / 总数: 1000 (准确率: 52.90%)

迭代次数 300, 损失 = 1.2978

正在验证集上检查准确率

正确数: 551 / 总数: 1000 (准确率: 55.10%)

迭代次数 400, 损失 = 1.1149

正在验证集上检查准确率

正确数: 577 / 总数: 1000 (准确率: 57.70%)

迭代次数 500, 损失 = 0.8144

正在验证集上检查准确率

正确数: 581 / 总数: 1000 (准确率: 58.10%)

迭代次数 600, 损失 = 1.3696

正在验证集上检查准确率

正确数: 598 / 总数: 1000 (准确率: 59.80%)

迭代次数 700, 损失 = 0.9641

正在验证集上检查准确率

正确数: 622 / 总数: 1000 (准确率: 62.20%)

迭代次数 0, 损失 = 0.9616

正在验证集上检查准确率

正确数: 635 / 总数: 1000 (准确率: 63.50%)

迭代次数 100, 损失 = 0.6734

正在验证集上检查准确率

正确数: 641 / 总数: 1000 (准确率: 64.10%)

迭代次数 200, 损失 = 0.9761

正在验证集上检查准确率

正确数: 672 / 总数: 1000 (准确率: 67.20%)

迭代次数 300, 损失 = 0.6526

正在验证集上检查准确率

正确数: 675 / 总数: 1000 (准确率: 67.50%)

迭代次数 400, 损失 = 0.7141

正在验证集上检查准确率

正确数: 677 / 总数: 1000 (准确率: 67.70%)

迭代次数 500, 损失 = 0.7723

正在验证集上检查准确率

正确数: 675 / 总数: 1000 (准确率: 67.50%)

迭代次数 600, 损失 = 0.8716

正在验证集上检查准确率

正确数: 680 / 总数: 1000 (准确率: 68.00%)

迭代次数 700, 损失 = 0.9408

正在验证集上检查准确率

正确数: 715 / 总数: 1000 (准确率: 71.50%)

迭代次数 0, 损失 = 0.6993

正在验证集上检查准确率

正确数: 693 / 总数: 1000 (准确率: 69.30%)

迭代次数 100, 损失 = 0.4297

正在验证集上检查准确率

正确数: 724 / 总数: 1000 (准确率: 72.40%)

迭代次数 200, 损失 = 0.4654

正在验证集上检查准确率

正确数: 718 / 总数: 1000 (准确率: 71.80%)

迭代次数 300, 损失 = 0.6587

正在验证集上检查准确率

正确数: 728 / 总数: 1000 (准确率: 72.80%)

迭代次数 400, 损失 = 0.9584

正在验证集上检查准确率

正确数: 722 / 总数: 1000 (准确率: 72.20%)

迭代次数 500, 损失 = 0.5561

正在验证集上检查准确率

正确数: 715 / 总数: 1000 (准确率: 71.50%)

迭代次数 600, 损失 = 0.6762

正在验证集上检查准确率

正确数: 718 / 总数: 1000 (准确率: 71.80%)

迭代次数 700, 损失 = 0.8128

正在验证集上检查准确率

正确数: 744 / 总数: 1000 (准确率: 74.40%)

迭代次数 0, 损失 = 0.5303

正在验证集上检查准确率

正确数: 734 / 总数: 1000 (准确率: 73.40%)

迭代次数 100, 损失 = 0.3088

正在验证集上检查准确率

正确数: 732 / 总数: 1000 (准确率: 73.20%)

迭代次数 200, 损失 = 0.3991

正在验证集上检查准确率

正确数: 736 / 总数: 1000 (准确率: 73.60%)

迭代次数 300, 损失 = 0.5014
正在验证集上检查准确率
正确数: 747 / 总数: 1000 (准确率: 74.70%)

迭代次数 400, 损失 = 0.4026
正在验证集上检查准确率
正确数: 746 / 总数: 1000 (准确率: 74.60%)

迭代次数 500, 损失 = 0.4788
正在验证集上检查准确率
正确数: 752 / 总数: 1000 (准确率: 75.20%)

迭代次数 600, 损失 = 0.4135
正在验证集上检查准确率
正确数: 735 / 总数: 1000 (准确率: 73.50%)

迭代次数 700, 损失 = 0.2226
正在验证集上检查准确率
正确数: 733 / 总数: 1000 (准确率: 73.30%)

迭代次数 0, 损失 = 0.1775
正在验证集上检查准确率
正确数: 742 / 总数: 1000 (准确率: 74.20%)

迭代次数 100, 损失 = 0.2983
正在验证集上检查准确率
正确数: 766 / 总数: 1000 (准确率: 76.60%)

迭代次数 200, 损失 = 0.3590
正在验证集上检查准确率
正确数: 756 / 总数: 1000 (准确率: 75.60%)

迭代次数 300, 损失 = 0.1307
正在验证集上检查准确率
正确数: 759 / 总数: 1000 (准确率: 75.90%)

迭代次数 400, 损失 = 0.1371
正在验证集上检查准确率
正确数: 744 / 总数: 1000 (准确率: 74.40%)

迭代次数 500, 损失 = 0.2208
正在验证集上检查准确率
正确数: 743 / 总数: 1000 (准确率: 74.30%)

迭代次数 600, 损失 = 0.1913
正在验证集上检查准确率
正确数: 762 / 总数: 1000 (准确率: 76.20%)

迭代次数 700, 损失 = 0.3318
正在验证集上检查准确率
正确数: 742 / 总数: 1000 (准确率: 74.20%)

迭代次数 0, 损失 = 0.0805
正在验证集上检查准确率
正确数: 770 / 总数: 1000 (准确率: 77.00%)

迭代次数 100, 损失 = 0.0897
正在验证集上检查准确率
正确数: 774 / 总数: 1000 (准确率: 77.40%)

迭代次数 200, 损失 = 0.0934
正在验证集上检查准确率
正确数: 764 / 总数: 1000 (准确率: 76.40%)

迭代次数 300, 损失 = 0.0537
正在验证集上检查准确率
正确数: 770 / 总数: 1000 (准确率: 77.00%)

迭代次数 400, 损失 = 0.1318
正在验证集上检查准确率
正确数: 769 / 总数: 1000 (准确率: 76.90%)

迭代次数 500, 损失 = 0.1051

正在验证集上检查准确率

正确数: 763 / 总数: 1000 (准确率: 76.30%)

迭代次数 600, 损失 = 0.0712

正在验证集上检查准确率

正确数: 772 / 总数: 1000 (准确率: 77.20%)

迭代次数 700, 损失 = 0.0667

正在验证集上检查准确率

正确数: 759 / 总数: 1000 (准确率: 75.90%)

迭代次数 0, 损失 = 0.0529

正在验证集上检查准确率

正确数: 754 / 总数: 1000 (准确率: 75.40%)

迭代次数 100, 损失 = 0.0449

正在验证集上检查准确率

正确数: 770 / 总数: 1000 (准确率: 77.00%)

迭代次数 200, 损失 = 0.0286

正在验证集上检查准确率

正确数: 761 / 总数: 1000 (准确率: 76.10%)

迭代次数 300, 损失 = 0.0304

正在验证集上检查准确率

正确数: 771 / 总数: 1000 (准确率: 77.10%)

迭代次数 400, 损失 = 0.0311

正在验证集上检查准确率

正确数: 765 / 总数: 1000 (准确率: 76.50%)

迭代次数 500, 损失 = 0.0992

正在验证集上检查准确率

正确数: 772 / 总数: 1000 (准确率: 77.20%)

迭代次数 600, 损失 = 0.0291

正在验证集上检查准确率

正确数: 778 / 总数: 1000 (准确率: 77.80%)

迭代次数 700, 损失 = 0.0284

正在验证集上检查准确率

正确数: 775 / 总数: 1000 (准确率: 77.50%)

迭代次数 0, 损失 = 0.0283

正在验证集上检查准确率

正确数: 767 / 总数: 1000 (准确率: 76.70%)

迭代次数 100, 损失 = 0.0172

正在验证集上检查准确率

正确数: 788 / 总数: 1000 (准确率: 78.80%)

迭代次数 200, 损失 = 0.0201

正在验证集上检查准确率

正确数: 783 / 总数: 1000 (准确率: 78.30%)

迭代次数 300, 损失 = 0.0458

正在验证集上检查准确率

正确数: 785 / 总数: 1000 (准确率: 78.50%)

迭代次数 400, 损失 = 0.0170

正在验证集上检查准确率

正确数: 784 / 总数: 1000 (准确率: 78.40%)

迭代次数 500, 损失 = 0.0035

正在验证集上检查准确率

正确数: 779 / 总数: 1000 (准确率: 77.90%)

迭代次数 600, 损失 = 0.0245

正在验证集上检查准确率

正确数: 791 / 总数: 1000 (准确率: 79.10%)

迭代次数 700, 损失 = 0.0125

正在验证集上检查准确率

正确数: 774 / 总数: 1000 (准确率: 77.40%)

迭代次数 0, 损失 = 0.0074

正在验证集上检查准确率

正确数: 775 / 总数: 1000 (准确率: 77.50%)

迭代次数 100, 损失 = 0.0116

正在验证集上检查准确率

正确数: 781 / 总数: 1000 (准确率: 78.10%)

迭代次数 200, 损失 = 0.0077

正在验证集上检查准确率

正确数: 787 / 总数: 1000 (准确率: 78.70%)

迭代次数 300, 损失 = 0.0057

正在验证集上检查准确率

正确数: 781 / 总数: 1000 (准确率: 78.10%)

迭代次数 400, 损失 = 0.0041

正在验证集上检查准确率

正确数: 780 / 总数: 1000 (准确率: 78.00%)

迭代次数 500, 损失 = 0.0074

正在验证集上检查准确率

正确数: 780 / 总数: 1000 (准确率: 78.00%)

迭代次数 600, 损失 = 0.0269

正在验证集上检查准确率

正确数: 783 / 总数: 1000 (准确率: 78.30%)

迭代次数 700, 损失 = 0.0125

正在验证集上检查准确率

正确数: 791 / 总数: 1000 (准确率: 79.10%)

迭代次数 0, 损失 = 0.0071

正在验证集上检查准确率

正确数: 784 / 总数: 1000 (准确率: 78.40%)

迭代次数 100, 损失 = 0.0060
正在验证集上检查准确率
正确数: 789 / 总数: 1000 (准确率: 78.90%)

迭代次数 200, 损失 = 0.0027
正在验证集上检查准确率
正确数: 789 / 总数: 1000 (准确率: 78.90%)

迭代次数 300, 损失 = 0.0056
正在验证集上检查准确率
正确数: 780 / 总数: 1000 (准确率: 78.00%)

迭代次数 400, 损失 = 0.0037
正在验证集上检查准确率
正确数: 779 / 总数: 1000 (准确率: 77.90%)

迭代次数 500, 损失 = 0.0082
正在验证集上检查准确率
正确数: 789 / 总数: 1000 (准确率: 78.90%)

迭代次数 600, 损失 = 0.0032
正在验证集上检查准确率
正确数: 788 / 总数: 1000 (准确率: 78.80%)

迭代次数 700, 损失 = 0.0033
正在验证集上检查准确率
正确数: 784 / 总数: 1000 (准确率: 78.40%)

8.1 描述你所做的工作

- 架构: 使用 `nn.Sequential` 构建了一个增强版卷积网络, 包含三个卷积块 (Conv -> ReLU -> BatchNorm / Pool), 然后展平并接两个全连接层。具体结构: Conv(3→64,5,pad=2) → ReLU → BN → Conv(64→64,3,pad=1) → ReLU → MaxPool(2) → Conv(64→128,3,pad=1) → ReLU → BN → MaxPool(2) → Flatten → FC(12888 → 512) → ReLU → FC(512 → 10)。
- 初始化: 对卷积层和线性层使用 Kaiming 正态初始化, 偏置初始化为零。
- 优化器: 使用 `torch.optim.SGD`, 学习率 `learning_rate` (单元中定义), 动量 0.9, 开启

Nesterov, 权重衰减 $5e-4$ 。

- 训练：调用 `train_part34(model, optimizer, epochs=10)` 进行 10 轮训练（该单元已执行）。
- 目标与说明：目标是在 10 轮内使验证集准确率达到 70%；所采用的增强结构（更多通道、BatchNorm、适当下采样和较大的中间全连接层）是为提高收敛速度与最终准确率而设计的。

答案：

8.2 测试集——仅运行一次

既然我们已经得到了满意的结果，现在就用我们的最终模型在测试集上进行测试（你应该将最终模型存储在 `best_model` 中）。思考一下这与你的验证集准确率相比如何。

```
[65]: best_model = model # 将训练好的最佳模型赋值给 best_model
      # 在测试集上检查最佳模型的准确率
      check_accuracy_part34(loader_test, best_model)
```

正在测试集上检查准确率

正确数：7752 / 总数：10000（准确率：77.52%）